

Notes Séance 1 – Timoté Policet

Language compilé :

Performant mais complexe à debug

GO, C++, C...

nécessite une bonne compréhension de la syntaxe

Language Interprété :

Moins performant mais plus clair car traduit

Python

Risque de fuite de mémoire

Les langages objets impliquent souvent une notion d'héritage qui est lourde.

Objectifs de go :

Bas niveau

Web

Mobile

-> Performances

Avantages Go :

Propose un système de composition (pas d'héritage, un objet peut avoir des caractéristiques à lui même sans les obtenir d'un parent)

Simplicité au développement (écriture/lecture)

Rapidité en performances (exécution/compilation)

Go routines (multithreading)

La concurrence

Définition : plusieurs threads (processus) qui accèdent à la même ressource mémoire

Pose des problèmes en termes de RAM;

Données corrompues

Données bloquées

Solution : processus tâches, multi-threading

Notes Séance 1 – Timoté Policet

Go routines

Plusieurs tâches multithreading

Outils utilisant ou développés en Go

Terraform

Kubernetes

Prometheus

Daemons systèmes

Agents DevOps

Code :

```
main.go
1  package main
2
3  import (
4      "fmt"
5      "math"
6      "time"
7  )
8
9  func main() {
10     // Hello, world !
11     fmt.Println("Hello, world !")
12
13     // 64 puissance 8
14     fmt.Println(math.Pow(64, 8))
15
16     current_time := time.Now().Local()
17
18     // Afficher la date et l'heure
19     fmt.Println(current_time.Date())
20
21     // Age
22     birth_date := time.Date(2003, time.April, 13, 12, 35, 0, 0, time.UTC)
23     age_year := current_time.Year() - birth_date.Year()
24     age_month := current_time.Month() - birth_date.Month()
25     age_day := current_time.Day() - birth_date.Day()
26     fmt.Println(age_year, int(age_month), age_day)
27 }
28
```

```
timote@MacBook-Pro-de-Timote swarm % go run main.go
Hello, world !
2.81474976710656e+14
2026 May 26
23 1 13
```

Explication des commandes

go run : Compile et exécute temporairement

go build : Compile et génère un fichier exécutable

Notes Séance 1 – Timoté Policet

Flags :

o : Nomme le fichier de sortie.

race : Détecte les conflits de mémoire.

v : Affiche la progression.

go build -ldflags "-s -w"

s -w : Supprime les symboles de débogage

Var : 0 problème de compilation

iota : un compteur automatique qui ne marche que dans les blocs const

On veut écrire des constantes qui se suivent. Au lieu d'écrire 0, 1, 2, 3... on peut le faire automatiquement avec iota

On ouvre un bloc const (...), iota démarre à 0.

À chaque nouvelle ligne, il s'incrémente de 1, tout seul.

Si tu laisses une ligne vide (juste le nom de la constante), Go répète la formule de la ligne du dessus, mais avec la nouvelle valeur d'iota.

Donc c'est un compteur qui s'incrémente tout seul dans un bloc const, et ensuite on peut faire des opérations dessus